CallAudit Pro

AI-Powered Customer Support Quality Auditor

End-to-End Project Documentation

Agile Development · 8 Weeks · 4 Milestones

Author	Srivani Tadaka
Project	AI Customer Support Quality Auditor
Version	1.0 — Final
Date	26 March 2026
Status	Completed

Table of Contents

1. **Project Overview**
2. Problem Statement & Objectives
3. Tech Stack & Architecture
4. Module Breakdown
5. Agile Milestones
   - Milestone 1 — Transcription & Ingestion
   - Milestone 2 — LLM Scoring Engine
   - Milestone 3 — RAG Pipeline & Dashboards
   - Milestone 4 — Alerts, Reports & Deployment
6. Vector Database Strategy
7. Dashboard Features
8. API Reference
9. How to Run the Project
10. Deployment Guide
11. Testing Guide
12. Known Limitations
13. Future Improvements
14. Glossary

1. Project Overview

CallAudit Pro is a GenAI-powered quality auditing platform built to review customer support calls and chats, assign performance scores, detect compliance violations, and suggest improvements in real time. The system was developed over 8 weeks following an Agile methodology, progressing through four clearly defined milestones from raw audio ingestion to a fully deployed cloud application.

The platform was designed with enterprise, BPO, and SaaS companies in mind — teams that handle high volumes of support interactions and need automated, consistent quality assessment without relying on manual review processes. By combining modern NLP techniques, retrieval-augmented generation, and large language model scoring, CallAudit Pro delivers what would otherwise take a human QA analyst hours — in under 60 seconds.

What the system does

- Takes a call recording (mp3, wav, m4a) or a text transcript as input
- Transcribes audio to text using Deepgram Nova-2 with high accuracy
- Searches a vector database for similar past calls as scoring reference
- Retrieves relevant compliance policy rules from the Knowledge Base
- Sends the enriched context to Groq LLaMA 3.3-70b for structured scoring
- Returns a full quality report: score, grade, violations, improvements, highlights
- Fires compliance alerts via in-app notifications and email if thresholds are breached
- Generates downloadable PDF and Excel reports
- Displays team-level analytics on a supervisor dashboard

2. Problem Statement & Objectives

The Problem

Customer support quality assurance is largely a manual process in most organisations. A supervisor or QA analyst listens to recorded calls, scores them on a rubric, flags violations, and writes feedback. This is time-consuming, inconsistent between reviewers, and only covers a small percentage of total calls due to capacity constraints. Teams miss critical compliance breaches, miss coaching opportunities, and have no real-time visibility into agent performance trends.

The Solution

An automated AI pipeline that can process every call, score it consistently against the same rubric, detect violations against known policy rules, and surface insights to supervisors in real time — removing the bottleneck of manual review while improving both coverage and consistency.

Core Objectives

1. Automate analysis of both chat transcripts and recorded calls
2. Score quality across five dimensions: empathy, professionalism, compliance, resolution effectiveness, and communication clarity
3. Detect compliance violations in real time with severity classification
4. Provide contextual feedback using RAG — comparing against past calls and policy documents
5. Deliver supervisor-level analytics through an interactive dashboard
6. Support deployment on cloud infrastructure accessible from anywhere

3. Tech Stack & Architecture

Full Technology Stack

Category	Technology	Purpose
LLM	Groq llama-3.3-70b-versatile	Call scoring and analysis
LLM Framework	LangChain	Prompt engineering and chain management
Transcription	Deepgram Nova-2	Audio-to-text conversion
Embeddings	BGE-small-en-v1.5	384-dimension semantic vectors
Vector DB 1	FAISS	Local in-memory semantic search
Vector DB 2	ChromaDB	Local persistent vector storage
Vector DB 3	Pinecone	Cloud vector database
Vector DB 4	Milvus	Enterprise scale (mock on Windows)
Web Framework	Flask	Backend API and main dashboard
WSGI Server	Waitress	Production-grade server (no reloader)
Analytics UI	Streamlit	Supervisor and cloud dashboards
Charts	Plotly	Interactive data visualizations
PDF Reports	ReportLab	Professional PDF generation
Excel Reports	OpenPyXL	Multi-sheet Excel export
Email Alerts	Gmail SMTP	Compliance notifications
Audio Capture	PyAudio	Microphone input for live monitoring
Environment	Python 3.13 / Windows	Development platform

System Architecture

The architecture follows a clean pipeline pattern. Every request flows through the same sequence regardless of whether it comes from a file upload, a live microphone, or a batch process.

```
Input (audio/text)
     ↓
Transcription Layer  (Deepgram Nova-2)
     ↓
RAG Enrichment       (ChromaDB / Pinecone)
  ├── Similar past calls retrieved
  └── Policy rules retrieved from KB
     ↓
LLM Scoring          (LangChain + Groq)
     ↓
Compliance Check     (Alert Engine)
     ↓
Output               (JSON / PDF / Excel / Dashboard)
```

4. Module Breakdown

The project is structured as a collection of focused modules, each responsible for one part of the pipeline. This separation makes it easy to swap components — for example, switching from ChromaDB to Pinecone requires changing one line in analyzer.py.

Project Structure

Folder / File	Description	Status
transcription/deepgram_processor.py	Audio to text using Deepgram	Complete
faiss_search/faiss_store.py	FAISS vector index — build, search, stats	Complete
faiss_search/embedder.py	BGE-small-en-v1.5 embedding model	Complete
Knowledge_base/kb_loader.py	14-entry policy knowledge base	Complete
Knowledge_base/kb_store.json	Stored KB entries (JSON)	Complete
vector_db/chroma_store.py	ChromaDB backend — local persistent	Complete
vector_db/pinecone_db.py	Pinecone backend — cloud storage	Complete
vector_db/milvus_store.py	Milvus backend — enterprise (mock)	Complete
rag_pipeline/rag_pipeline.py	Unified RAG pipeline — 4 backends	Complete
llm/langchain_scorer.py	LangChain + Groq scoring engine	Complete
llm/batch_scorer.py	Batch processing for multiple files	Complete
realtime/audio_capture.py	Microphone audio capture in chunks	Complete
realtime/stream_transcriber.py	Deepgram chunk-by-chunk transcription	Complete
realtime/live_monitor.py	Orchestrates live call monitoring	Complete
realtime/alert_engine.py	Compliance violation detection + email	Complete
reports/pdf_report.py	ReportLab PDF report generator	Complete
reports/excel_report.py	OpenPyXL Excel export (5 sheets)	Complete
analyzer.py	Main analysis entry point	Complete
app.py	Flask server — all API routes	Complete
app_cloud.py	Unified Streamlit cloud app (4 pages)	Complete
streamlit_app.py	Supervisor analytics dashboard	Complete
templates/index.html	Flask main dashboard UI	Complete
templates/live_monitor.html	Flask live monitor page	Complete

5. Agile Milestones

The project was executed across 8 weeks in 4 two-week sprints. Each sprint had a clear objective, defined tasks, and measurable acceptance criteria.

Milestone 1 — Transcription & Data Ingestion (Weeks 1–2)

Objective	Process customer support chats and convert audio call recordings to structured text ready for analysis.

Tasks Completed

- Integrated Deepgram Nova-2 API for audio transcription
- Built deepgram_processor.py with retry logic (3 attempts), file format support (mp3, wav, m4a, flac, mp4), and error handling
- Created sample dataset with 10 call recordings and 10 chat transcripts covering diverse scenarios
- Defined JSON schema for all scored results — 25+ fields covering scores, violations, improvements, metadata
- Built text normalisation to handle both agent/customer labelled and unlabelled transcripts
- Achieved 95%+ transcription accuracy on test audio

Acceptance Criteria Met

- Audio files successfully transcribed to text ✓
- Chat transcripts loaded and parsed correctly ✓
- Schema defined and validated ✓

Note: Deepgram Nova-2 was chosen over OpenAI Whisper (specified in brief) due to faster inference, higher accuracy, and a simpler REST API with no local model download required.

Milestone 2 — LLM Scoring Engine (Weeks 3–4)

Objective	Build an automated scoring pipeline that uses a large language model to evaluate agent quality across multiple dimensions.

Tasks Completed

- Built original Groq scoring engine in llm/scoring_engine.py
- Replaced with LangChain integration in llm/langchain_scorer.py using PromptTemplate | ChatGroq | StrOutputParser chain
- Designed structured scoring prompt that forces the LLM to return valid JSON every time
- Implemented scoring for five quality dimensions: empathy (0-10), professionalism (0-10), compliance (0-10), resolution effectiveness (0-10), communication clarity (0-10)
- Added customer satisfaction metrics: sentiment, frustration level, stability, rating out of 5
- Added agent quality metrics: language clarity (0-20), professionalism (0-20), time efficiency (0-20), response efficiency (0-20), empathy score (0-10)
- Violations detection with type, severity (critical/high/medium/low), verbatim quote, and explanation
- Coaching improvements with area, suggestion, and example phrases
- Model confidence metrics: precision, recall, F1 score
- Built batch_scorer.py for processing all sample files in bulk

Acceptance Criteria Met

- LLM scoring functional across all five dimensions ✓
- Quality scores generated with grade A–F ✓
- Violations detected with severity classification ✓
- Improvements and highlights returned ✓

Note: Groq llama-3.3-70b-versatile was used instead of GPT-4 (specified in brief) because Groq provides 10-20x faster inference at zero cost, which is critical for real-time scoring.

Milestone 3 — RAG Pipeline & Dashboards (Weeks 5–6)

Objective	Add contextual retrieval using vector databases so the LLM can score calls relative to past examples and policy rules, and build supervisor-level dashboards.

RAG Pipeline Tasks

- Built FAISS vector store with BGE-small-en-v1.5 embeddings (384 dimensions, cosine similarity)
- Created Knowledge Base with 14 entries across 3 categories: 7 policy rules, 3 quality rubrics, 4 best practices
- Built ChromaDB backend — two collections (scored_calls, knowledge_base), persistent local storage

- Built Pinecone backend — two namespaces, serverless AWS us-east-1, cloud storage
- Built Milvus backend — mock implementation for Windows (milvus-lite not supported on Windows)
- Created unified RAGPipeline class supporting all four backends with one-line switching
- Integrated LangChain with RAG enrichment — similar calls and policy rules injected into every prompt

Dashboard Tasks

- Built Flask main dashboard (index.html) — upload, score, view all results
- Built Streamlit supervisor dashboard (streamlit_app.py) — KPIs, charts, leaderboard, violations table
- Built unified Streamlit cloud app (app_cloud.py) — 4 pages: Dashboard, Live Analysis, Supervisor, Reports
- Added real-time polling and transcript display on the live monitor page

Acceptance Criteria Met

- RAG pipeline operational across 4 vector DB backends ✓
- LangChain + Pinecone integration complete ✓
- Streamlit supervisor dashboard with analytics ✓
- Flask main dashboard with upload and scoring ✓

Milestone 4 — Alerts, Reports & Deployment (Weeks 7–8)

Objective	Enable real-time compliance alerts, generate downloadable reports, build live call monitoring, and prepare for cloud deployment.

Alerts

- Built AlertEngine class with score-based and violation-based triggers
- CRITICAL alert fires when overall score < 40 or violation severity = critical
- HIGH alert fires when violation severity = high
- WARNING alert fires when score is between 40 and 60
- In-app alerts displayed on dashboard as colour-coded cards
- Email alerts via Gmail SMTP using App Password authentication
- HTML email with score, grade, violation list, and AI summary

Reports

- PDF reports generated with ReportLab — score cards, dimension bars, violations, improvements, summary
- Excel export with 5 sheets: Summary KPIs, All Calls, Violations, Improvements, Agent Quality
- JSON raw export available for all scored calls
- Download buttons on both Flask and Streamlit dashboards

Live Monitor

- AudioCapture class captures microphone input in 5-second WAV chunks
- StreamTranscriber sends chunks to Deepgram and accumulates transcript
- LiveMonitor orchestrates capture, transcription, periodic scoring, and alerts
- Live monitor page at /live shows real-time transcript, score, and alerts
- Manual transcript input available as fallback for cloud environments

Deployment

- Configured Waitress as production WSGI server (no auto-reloader issues)
- Unified Streamlit cloud app built for Streamlit Community Cloud deployment
- Render.com deployment configured via Procfile
- Environment variables for all API keys — no secrets in code

Acceptance Criteria Met

- Compliance alerts firing in-app and via email ✓
- PDF reports downloadable per call ✓
- Excel export with all calls ✓
- Live monitor working with microphone input ✓
- System deployed and accessible ✓

6. Vector Database Strategy

One of the differentiating design decisions in CallAudit Pro was implementing four different vector database backends, all accessible through a single unified interface. This was done to demonstrate understanding of the trade-offs between different storage approaches.

Database	Type	Best Use Case
FAISS	Local in-memory	Fast local development, no setup required
ChromaDB	Local persistent	Development with data persistence across restarts
Pinecone	Cloud serverless	Production deployment, accessible from anywhere
Milvus	Enterprise server	High-scale production with millions of vectors

Switching between backends is a single configuration change in analyzer.py:

```
BACKEND = os.getenv('VECTOR_BACKEND', 'chromadb')
_pipeline = RAGPipeline(backend=BACKEND)
```

For local development, ChromaDB is the default because it persists data between restarts. For cloud deployment, Pinecone is used because it stores data on Pinecone's servers and is accessible from any cloud environment without local storage.

Each database stores two separate collections or namespaces: scored_calls (past call results used as grading references) and knowledge_base (14 policy rules used for compliance checking). This separation ensures that search results are always from the correct category.

7. Dashboard Features

Flask Main Dashboard (/)

The primary interface for uploading and analysing individual calls. Built with pure HTML, CSS, and vanilla JavaScript — no frontend framework required.

- File upload with drag-and-drop support (audio and transcript formats)
- Analyse & Score button with spinner and busy state
- Overview cards: overall score, customer satisfaction with star rating, call outcome and grade
- Customer satisfaction panel: sentiment, sentiment score bar, emotional stability, frustration level and reason
- Agent quality panel: five dimension bars (language clarity, professionalism, time efficiency, response efficiency, empathy)
- Dimension score cards with icons and animated progress bars
- Classification confidence metrics: precision, recall, F1 score, confidence
- AI summary and detected issue
- Violations panel with severity-coloured left borders and verbatim quotes
- Coaching improvements with example phrases
- Compliance alerts section (shows when alerts are triggered)
- Transcript viewer with Agent/Customer colour coding
- Highlights section for positive performance
- Batch table showing all scored conversations with stats strip
- Per-row PDF download link
- Excel export button

Streamlit Cloud App (app_cloud.py)

A unified four-page Streamlit application designed for cloud deployment.

- Page 1 — Dashboard: upload audio or transcript, full scoring result, PDF download
- Page 2 — Live Analysis: start/stop microphone monitoring, manual transcript paste, real-time scoring
- Page 3 — Supervisor: KPI cards, score distribution, grade breakdown, dimension averages, score trend, violations pie, leaderboard tabs, full calls table
- Page 4 — Reports: Excel generation, PDF per call, JSON export

Streamlit Supervisor Dashboard (streamlit_app.py)

Standalone supervisor analytics dashboard for team-level oversight.

- Sidebar filters: grade, outcome, score range
- Six KPI cards: total calls, average score, pass rate, violations, resolved, grade F count
- Score distribution histogram
- Grade breakdown bar chart
- Dimension averages horizontal bar chart
- Score trend line chart with pass threshold
- Violations by severity pie chart
- Outcome breakdown pie chart
- Worst 5 / Best 5 leaderboard tabs
- Full scored calls dataframe with progress bars
- All violations detail table

Live Monitor Page (/live)

Real-time call monitoring interface served by Flask.

- Start / Stop buttons with live timer
- Live transcript panel updating every 5 seconds
- Live score panel with dimension bars
- Compliance alerts panel with timestamp
- AI summary updating after each scoring cycle

8. API Reference

The Flask application exposes the following REST API endpoints:

Endpoint	Method	Description
GET /	GET	Serve main dashboard HTML
POST /analyze_ajax	POST	Upload file, transcribe, score, return JSON result
GET /results	GET	Return all scored results as JSON array
GET /download_pdf/<filename>	GET	Generate and download PDF for a scored call
GET /download_excel	GET	Generate and download Excel of all calls
GET /live	GET	Serve live monitor HTML page
POST /live/start	POST	Start microphone capture and monitoring
POST /live/stop	POST	Stop monitoring, return transcript
GET /live/status	GET	Poll current transcript, score, alerts

analyze_ajax Request

Accepts multipart/form-data with a file field. Supported formats: .mp3, .wav, .m4a, .flac, .mp4, .txt, .json

analyze_ajax Response

Returns a JSON object with the following key fields:

- grade — letter grade A through F
- overall_score — integer 0-100
- call_outcome — Resolved / Unresolved / Escalated / Callback Required
- was_resolved — boolean
- sentiment — positive / negative / neutral / mixed
- satisfaction — object with rating, sentiment_score, emotional_stability, customer_frustration, frustration_reason

- agent_quality — object with language_clarity, professionalism, time_efficiency, response_efficiency, empathy_score, bias_detected, calmed_customer
- dimension_scores — object with empathy, professionalism, compliance, resolution_effectiveness, communication_clarity
- model_metrics — object with precision, recall, f1_score, confidence
- violations — array of violation objects (type, severity, quote, explanation)
- improvements — array of improvement objects (area, suggestion, example)
- highlights — array of positive observation strings
- compliance_alerts — array of triggered alert objects
- summary — two-sentence AI summary string
- issue_detected — main issue string
- _saved_filename — filename of the saved JSON result for PDF generation

9. How to Run the Project

Prerequisites

- Python 3.10 or higher
- Virtual environment created at project root
- API keys for Groq, Deepgram, and Pinecone
- Gmail account with App Password for email alerts

Step 1 — Clone and set up

```
git clone https://github.com/Srivanitadaka/AI--Customer-Support-Quality-Auditor
cd AI--Customer-Support-Quality-Auditor
python -m venv venv
venv\Scripts\activate   # Windows
```

Step 2 — Install dependencies

```
pip install -r requirements.txt
```

Step 3 — Configure environment

Create config/.env with the following:

```
GROQ_API_KEY=your_groq_key
DEEPGRAM_API_KEY=your_deepgram_key
PINECONE_API_KEY=your_pinecone_key
PINECONE_INDEX=callaudit
VECTOR_BACKEND=chromadb
ALERT_EMAIL_FROM=your_gmail
ALERT_EMAIL_PASSWORD=your_app_password
ALERT_EMAIL_TO=supervisor_gmail
```

Step 4 — Build vector database

```
python knowledge_base\kb_loader.py
python vector_db\chroma_store.py --build
python vector_db\chroma_store.py --stats
```

Step 5 — Run Flask app

```
python app.py
```

Open http://127.0.0.1:5000 in your browser.

Step 6 — Run Streamlit apps (separate terminals)

```
streamlit run app_cloud.py      # unified cloud app
streamlit run streamlit_app.py  # supervisor dashboard
```

10. Deployment Guide

Streamlit Community Cloud (Supervisor + Cloud App)

7. Push project to GitHub
8. Go to share.streamlit.io and sign in with GitHub
9. Click New app → select repository → set main file to app_cloud.py
10. Go to App settings → Secrets and add all API keys in TOML format
11. Set VECTOR_BACKEND = pinecone in secrets
12. Click Deploy — app will be live at https://yourapp.streamlit.app

Render.com (Flask + Live Monitor)

13. Ensure Procfile exists: web: waitress-serve --host=0.0.0.0 --port=$PORT app:app
14. Go to render.com → New → Web Service → connect GitHub repo
15. Set Build Command: pip install -r requirements.txt
16. Set Start Command: waitress-serve --host=0.0.0.0 --port=$PORT app:app
17. Add all environment variables in the Environment tab
18. Set VECTOR_BACKEND = pinecone
19. Click Deploy — app will be live at https://yourapp.onrender.com

Important: ChromaDB, FAISS, and Milvus cannot be used in cloud deployments because they rely on local file storage. Pinecone is the only vector database that works on cloud infrastructure because it stores data on Pinecone's own servers.

11. Testing Guide

Test vector database components

```
python vector_db\chroma_store.py --build
python vector_db\chroma_store.py --stats
python vector_db\chroma_store.py --search "agent refused escalation"
python vector_db\pinecone_db.py --build
python vector_db\pinecone_db.py --stats
```

Test RAG pipeline

```
python rag_pipeline\rag_pipeline.py
```

Speak or edit the test transcript at the bottom of the file to test with different scenarios.

Test LangChain scorer

```
python llm\langchain_scorer.py
```

This runs a test transcript through the full LangChain chain and prints a structured result.

Test live monitor

```
python realtime\audio_capture.py
python realtime\stream_transcriber.py
python realtime\live_monitor.py
```

Test email alerts

```
python -c "
import smtplib, os
from dotenv import load_dotenv
load_dotenv('config/.env')
FROM = os.getenv('ALERT_EMAIL_FROM')
PASS = os.getenv('ALERT_EMAIL_PASSWORD')
TO   = os.getenv('ALERT_EMAIL_TO')
with smtplib.SMTP_SSL('smtp.gmail.com', 465) as s:
    s.login(FROM, PASS)
    s.sendmail(FROM, TO, 'Subject: Test\n\nWorks!')
print('Email sent')"
```

12. Known Limitations

Limitation	Explanation / Workaround
Milvus on Windows	milvus-lite does not support Windows. The current implementation uses a pure Python mock with identical API. On Linux/Mac deployment, swap get_client() to use real MilvusClient.
deepgram-sdk venv conflict	If two virtual environments exist (Desktop and Downloads), pip may install to the wrong one. Always use the full path: venv\Scripts\pip.exe install deepgram-sdk --force-reinstall
Flask auto-reloader	Flask's watchdog reloader detects Python library file changes mid-request and restarts the server. Fixed by using Waitress instead of the Flask dev server.
ChromaDB on cloud	ChromaDB stores data locally and cannot be used in cloud deployments. Pinecone must be used for any cloud-hosted instance.
Live mic on Streamlit Cloud	Browser microphone access through Streamlit Cloud has limitations. The manual transcript input is provided as a reliable fallback for cloud environments.
Free tier cold starts	Render.com free tier sleeps after 15 minutes of inactivity. The first request after sleep takes 30-60 seconds to wake the server.

13. Future Improvements

The following enhancements have been identified for future development sprints:

Short Term (Sprint 5)

- Agent name tracking — associate each call with a named agent for individual performance reports
- Time-series analysis — track agent improvement or degradation over time
- Multi-language support — extend Deepgram transcription to support regional Indian languages
- Webhook notifications — push alerts to Slack, Teams, or WhatsApp via webhook

Medium Term (Sprint 6–7)

- Role-based access — separate login for agents (see own reports), supervisors (see team), admins (full access)
- Custom scoring rubrics — allow organisations to define their own quality criteria and weights
- Automated coaching recommendations — generate weekly coaching plans per agent based on trends
- Real-time streaming transcription — use Deepgram's WebSocket streaming instead of chunk-based REST

Long Term (Sprint 8+)

- Docker containerisation — package entire application with all dependencies for one-command deployment
- Kubernetes scaling — support hundreds of concurrent calls in large enterprise environments
- Fine-tuned LLM — fine-tune a smaller model on domain-specific call data for faster, cheaper scoring
- Integration with CRM systems — sync results with Salesforce, Zendesk, or Freshdesk

14. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation. A technique where relevant documents are retrieved from a vector database and injected into the prompt before calling an LLM.
LLM	Large Language Model. A deep learning model trained on vast text data. This project uses llama-3.3-70b hosted by Groq.
Vector Database	A database that stores high-dimensional numerical representations (embeddings) of text, enabling semantic similarity search.
Embedding	A numerical representation of text as a dense vector. Similar text produces similar vectors. This project uses 384-dimensional BGE embeddings.
Cosine Similarity	A metric used to measure how similar two vectors are, regardless of their magnitude. Used for finding similar past calls.
ChromaDB	An open-source vector database that stores embeddings and metadata locally on disk.
Pinecone	A cloud-native vector database that stores embeddings on remote servers, enabling access from any cloud environment.
FAISS	Facebook AI Similarity Search. A library for efficient similarity search stored entirely in RAM.
LangChain	A framework for building applications with LLMs. Used here for the PromptTemplate → ChatGroq → StrOutputParser chain.
Groq	An AI inference company providing extremely fast LLM inference. Used to run llama-3.3-70b in under 2 seconds.
Deepgram	A speech recognition API. Nova-2 model used for transcribing call recordings.
Streamlit	A Python library for building interactive data applications. Used for supervisor and cloud dashboards.
Waitress	A production-grade Python WSGI server. Used instead of Flask's development server to avoid auto-restart issues.

Term	Definition
WSGI	Web Server Gateway Interface. The standard Python interface between web frameworks (Flask) and web servers (Waitress).
Compliance Alert	An automated notification triggered when a call score falls below a threshold or when a policy violation is detected.
BGE	BAAI General Embedding. A family of text embedding models from the Beijing Academy of AI. BGE-small-en-v1.5 produces 384-dimensional vectors.